Checksum repair and pre-download simulation

How Enhancements 5 and 7 work in Plexo, and how to prove it in a five-minute demo. CS2403 Computer Networks, 7 November 2026.

`5/5 checksum checks pass · 9/9 simulation checks pass · 2 bad blocks found out of 16 · 512 KiB rewritten · 0 bytes re-fetched on a clean file · stops after 3 rounds`

Enhancement 5 is no longer only a design. Plexo now checks every finished HTTP download against an MD5, SHA-1 or SHA-256 that the user types in or the page publishes. When the file is wrong, it finds the blocks a faulty network damaged and fetches only those again over the best ranked network, giving up after three rounds so the user can decide. Enhancement 7 was already built; it gained three fixes and five new checks. Both are proven by one command, npm test, which builds the failure on purpose and shows it being caught.

1 What changed in the code

Nine places changed. Three files are new; the rest gained a few lines each. Every path is relative to the plexo folder.

File	What it does now
`src/main/download/checksum.ts (new)`	Hashes the file, compares blocks network by network, runs at most three repair rounds
`src/shared/checksum.ts (new)`	Recognises MD5, SHA-1 or SHA-256 from the length of what the user pastes
`src/main/download/downloadManager.ts`	Runs the check just before a file is published; ranks networks for repair; Publish unverified
`src/main/download/downloadFile.ts`	Writes a repaired block back at its own offset
`src/main/download/learning/store.ts`	Reads a network's learned throughput without creating a new profile
`NewDownloadDialog.tsx, ContentDetailsDialog.tsx`	New Checksum field; a hash published by an indexed page is passed on
`ChecksumPanel.tsx and three screens`	Shows Hashing, Repairing, Verified or Mismatch, with Retry repair and Publish unverified
`simulationEngine.ts, PreDownloadSimulationDialog.tsx`	Shares add up to exactly 100 %, real fastest path, assumptions note
`scripts/verify-checksum.test.mts (new), verify-simulation.test.mts`	The proof: run by npm test

Table 1. Where Enhancement 5 lives and what the Enhancement 7 fixes touched. Source: git diff of the plexo folder, 7 November 2026.

2 Checksum verification catches wrong bytes before the file is published

The failure base Plexo cannot see

Base Plexo checks every answer it gets. The status must be 206, the Content-Range must start where it asked, and the body must be the right length. A link that returns the right length with the wrong bytes passes all three. When one file comes from several networks at once, one bad link (a flaky hotspot, a proxy that rewrites content, a broken cache) can spoil a few blocks while the rest is fine. Nothing in the base app notices, and the file is published as complete.

How it works, in five steps

1. Get the expected checksum. The user pastes one into the new Checksum field of the New Download dialog, and the length tells the algorithm: 32 hex characters is MD5, 40 is SHA-1, 64 is SHA-256. From Discover, the hash an indexed page publishes is passed on by itself.
2. Hash the finished file. After the last block, Plexo streams the staging file through Node's built-in crypto, so even a large file never has to fit in memory. A match marks the download Verified and publishes it, with no extra traffic.
3. Pick a suspect. On a mismatch, the lowest ranked network that delivered blocks is suspected first. The ranking uses the learning store's per-network throughput for this host, then the bytes each network delivered in this run, then fewer retries.
4. Compare its blocks. Every block the suspect delivered is fetched again over the best ranked other network. Both copies are hashed, and only blocks whose hashes differ are written back over the bad ones.
5. Hash again, for at most three rounds. If the file still does not match, the next suspect is checked. After three rounds the download stops on Checksum mismatch with two buttons, Retry repair and Publish unverified. A wrong file is never published unless the user chooses it.

Why it checks one network at a time

HTTP gives no checksum per block, so the local file alone cannot say which block is wrong. What Plexo does know is which network delivered each block, because every block records its bytes by interface. Checking one network's share at a time means a clean network is never downloaded twice. In the test, the faulty Wi-Fi path delivered half the file, so half the file is fetched again and only the two damaged blocks, 512 KiB, are rewritten.

3 The simulation gives the answer before any byte moves

Press Simulate & Download on an indexed item and Enhancement 7 shows the combined speed, the speed-up over the fastest single path, the estimated time, each network's share of chunks, and whether the disk can hold the file while it is assembled. The model is plain arithmetic on assumed speeds, and the dialog now says so in one line under the numbers. The table repeats the worked example that the test checks to the byte.

Quantity	Rule	Ethernet + Wi-Fi, 10 GiB file
Assumed speed	Default for the adapter kind, capped by the user's limit	`60 MB/s and 28 MB/s`
Combined speed	Sum of the speeds × 0.92	`80.96 MB/s`
Fastest single path	Largest single speed	`60 MB/s`
Speed-up	Combined ÷ fastest, one decimal	`1.3×`
Estimated time	File size ÷ combined speed, rounded up	`127 s (2 min 7 s)`
Chunk share	Speed ÷ sum, largest-remainder rounding	`68 % and 32 %`
Disk needed	File size × 1.08 while assembling	`10.8 GiB`

Table 2. Every value in the right-hand column is asserted by scripts/verify-simulation.test.mts (check 4b). Source: constants in src/renderer/src/utils/simulationEngine.ts.

Three fixes made the numbers honest

The fastest single path used to start from a 20 MB/s default, so a lone network capped at 1 MB/s was compared with 20 MB/s and its single-path time came out twenty times too short. It now starts from zero. Rounding each share on its own could give 99 % or 101 % with three equal networks; largest-remainder rounding makes the shares add up to exactly 100 % and the bytes to the file size. The third fix is the assumptions note itself, which names the four default speeds and says nothing has been measured yet.

4 One command proves both features in front of the professor

The checksum test builds the failure on purpose. It starts a real HTTP server on 127.0.0.1 holding a 4 MiB file split into 16 blocks of 256 KiB. Two paths reach it. The eth path answers honestly. The wifi path answers 206 with the correct length but flips eight bytes inside blocks 3 and 9, the exact fault base Plexo cannot see. The file is assembled by alternating the two paths, as the scheduler does, and then handed to verifyAndRepair, the same function the download manager calls before it publishes a file.

The commands to run

```
cd plexo
node node_modules/.bin/jiti scripts/verify-checksum.test.mts
node node_modules/.bin/jiti scripts/verify-simulation.test.mts
npm test # every suite, these two included
```

What each line of the output proves

Output line	What it proves
`[PASS] 0`	MD5, SHA-1 and SHA-256 are recognised from a pasted string, in any case and with stray spaces

`[PASS] 1 Negative control`	The damaged file passes every status, length and range check, yet its SHA-256 is wrong. Without Enhancement 5 it would have been published as complete
`[PASS] 2 Clean file`	A good file is verified on the first hash: 0 repair rounds, 0 bytes fetched again
`[PASS] 3 Damaged file repaired`	Blocks #3 and #9 found, wifi named as the bad path, 2 MiB fetched again over eth only, 512 KiB rewritten, final SHA-256 matches
`[PASS] 4 Lying network`	With only a lying path left, repair stops after exactly 3 rounds and the file is not published
`[PASS] 4b to 4e (simulation)`	Hand arithmetic of Table 2, the speed-limit cap, the real fastest path, shares that add up to 100 %

Table 3. The five checksum checks and the new simulation checks, as printed by the two test scripts. Source: run on 7 November 2026, all passing.

Showing it in the running app

Start the app with npm run dev. Paste a link to a file whose SHA-256 you know (a Linux ISO page lists one), paste the digest into the new Checksum field, and start. The download screen shows "Hashing the file (SHA-256)" and then "SHA-256 verified". Now start the same file with one character of the digest changed. It ends on Checksum mismatch with Retry repair and Publish unverified, and the file stays as a .plexo partial, unpublished. Use a small file for this second run: before giving up, Plexo spends its three rounds fetching blocks again. For Enhancement 7, open any item in Discover and press Simulate & Download; the assumptions line sits under the numbers.

5 What the tests do not cover

The repair test drives the real checksum module over real HTTP, but the two paths are two URLs on one machine, not two network cards, and the Electron window is not part of it. The in-app parts (the Checksum field, the status panel and its two buttons) are type-checked and lint-clean but have no automated UI test, so show them by hand as described above. Repair needs a server that accepts Range requests; without one, a mismatch is reported at once with no repair rounds. The simulation's speeds remain assumptions, not measurements, and the dialog says so.